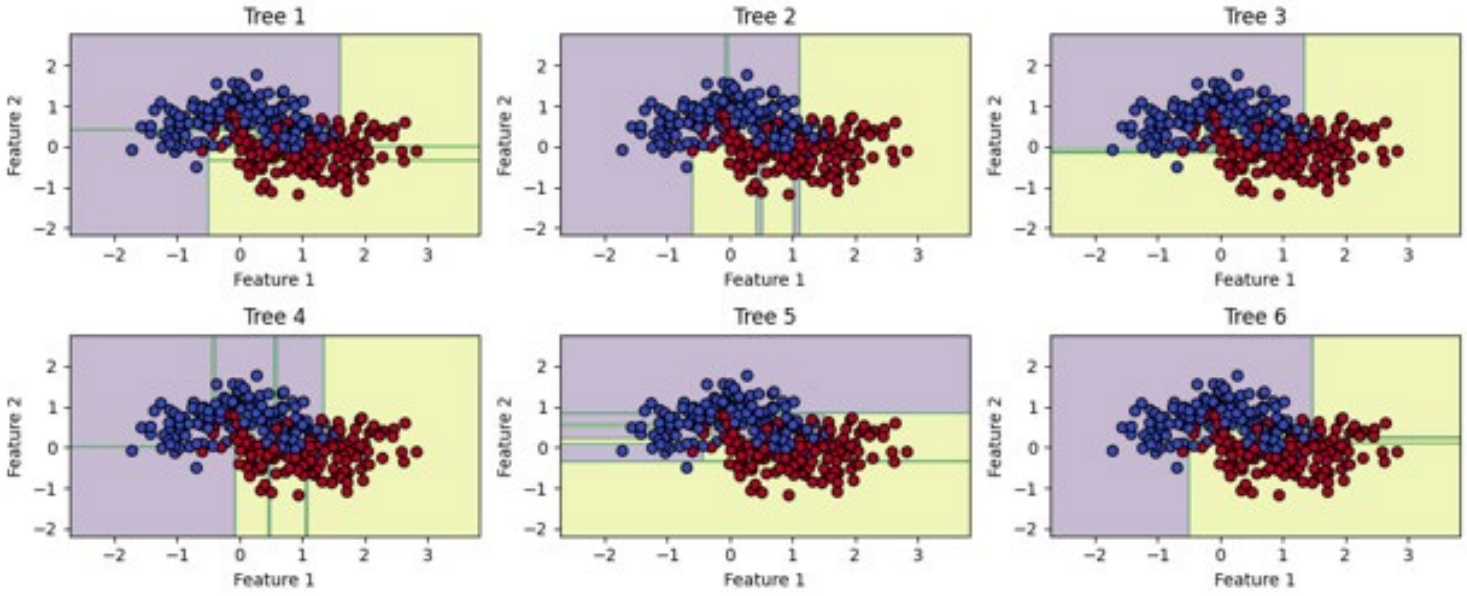
Chapter 13:

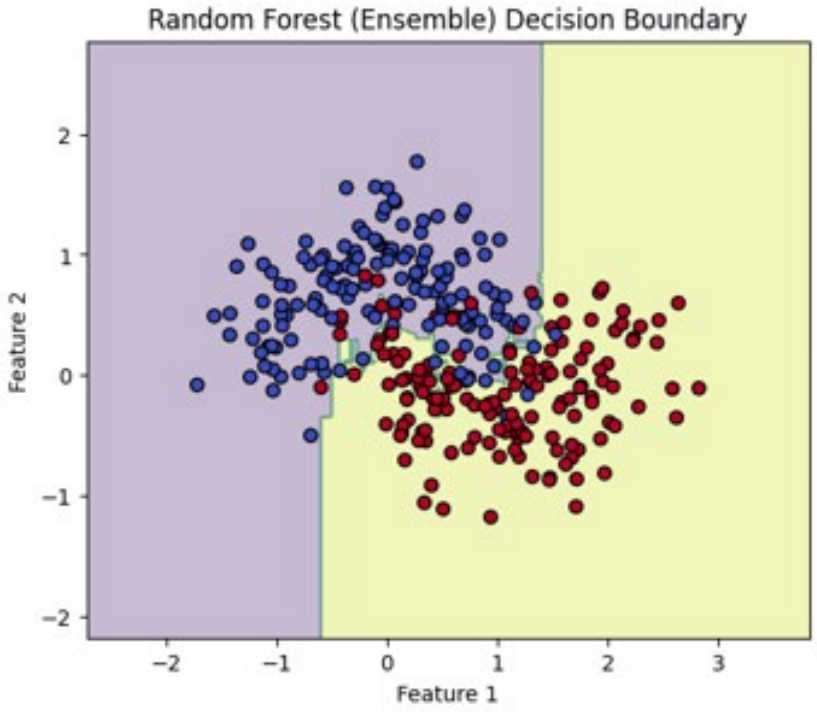
Ensemble Methods

Ensemble Methods

- Multiple learners are **combined** in ensemble learning to enhance prediction performance.
- Stated differently, an ensemble model generates predictions that are **more accurate** than those of a single model alone by combining multiple distinct models.
- The particular model(s) employed in ensemble algorithms are referred to as the **base learner**, base model, and, in certain situations, base estimator.
- Learners or models that perform marginally better than random guessing is considered **weak.**
- **Strong** models or learners, on the other hand, produce exceptional prediction performance.

Ensemble Methods





Key Concepts of Ensemble Methods

1. **Diversity:** The models in an ensemble should be diverse, meaning they should make different kinds of errors. This diversity ensures that the weaknesses of one model are compensated by the strengths of others.
2. **Combination:** The predictions of individual models are combined in some way, such as averaging (for regression) or voting (for classification).

Advantages of Ensemble Methods:

1. **Improved Accuracy:** Often, combining models produces better results than using just one.
2. **Robustness:** Enhances generality and lowers the chance of overfitting.
3. **Handles Complex Data:** Can capture complex patterns in data by combining different models.

Disadvantages of Ensemble Methods:

1. **Computational Cost:** Training and combining multiple models can be computationally expensive.
2. **Complexity:** Ensembles are harder to interpret compared to single models.
3. **Overfitting Risk:** If not properly tuned, some ensemble methods (e.g., boosting) can overfit the training data.

When to Use Ensemble Methods:

1. When you have a diverse set of models that perform well individually but make different errors.
2. When you want to improve the stability and accuracy of predictions.
3. When computational resources are sufficient to train and combine multiple models.

Bias-variance tradeoff

Bias:

- ✓ Describes the mistake that arises when a simple model is used to approximate a complicated real-world problem.
- ✓ A model that exhibits high bias may underfit the data, failing to identify the underlying patterns.
- ✓ Example: A linear model used to fit data that has a nonlinear relationship.

Variance:

- ✓ Variance is the inaccuracy brought about by the model's sensitivity to small changes in the training set.
- ✓ High variance can cause a model to overfit the data, meaning it captures noise or random fluctuations in the training data instead of the underlying pattern.
- ✓ Example: A very complex model (e.g., a high-degree polynomial) that fits the training data perfectly but performs poorly on unseen data.

Irreducible Error:

- ✓ This is the error that cannot be reduced by any model, as it is inherent in the data (e.g., noise).

Bias-variance tradeoff

A well-known issue in machine learning is the bias-variance tradeoff.

The average discrepancy between expected and actual numbers is known as **bias**.

- ✓ On a training dataset, a model's prediction accuracy decreases with increasing bias.
- ✓ High bias is synonymous with high training error.
- ✓ Optimization denotes efforts to lessen bias.

Variance quantifies the discrepancy between forecasts across different model realizations.

- ✓ A model's ability to forecast on unknown data decreases with increasing variation.
- ✓ In testing and validation, high variance means high error.
- ✓ Attempts to minimize variance are referred to as generalization.

Thus, model accuracy on training and test data is inversely represented by bias and variance.

Mathematical Representation

A model's overall inaccuracy can be broken down into three parts:

$$Error = Bias^2 + Variance + Irreducible\ Error$$

- $Bias^2$: Evaluates the average difference between the model's predictions and the actual values.
- Variance: Measures how much the model's predictions fluctuate for different training datasets.
- Irreducible Error: The noise in the data that cannot be modelled.

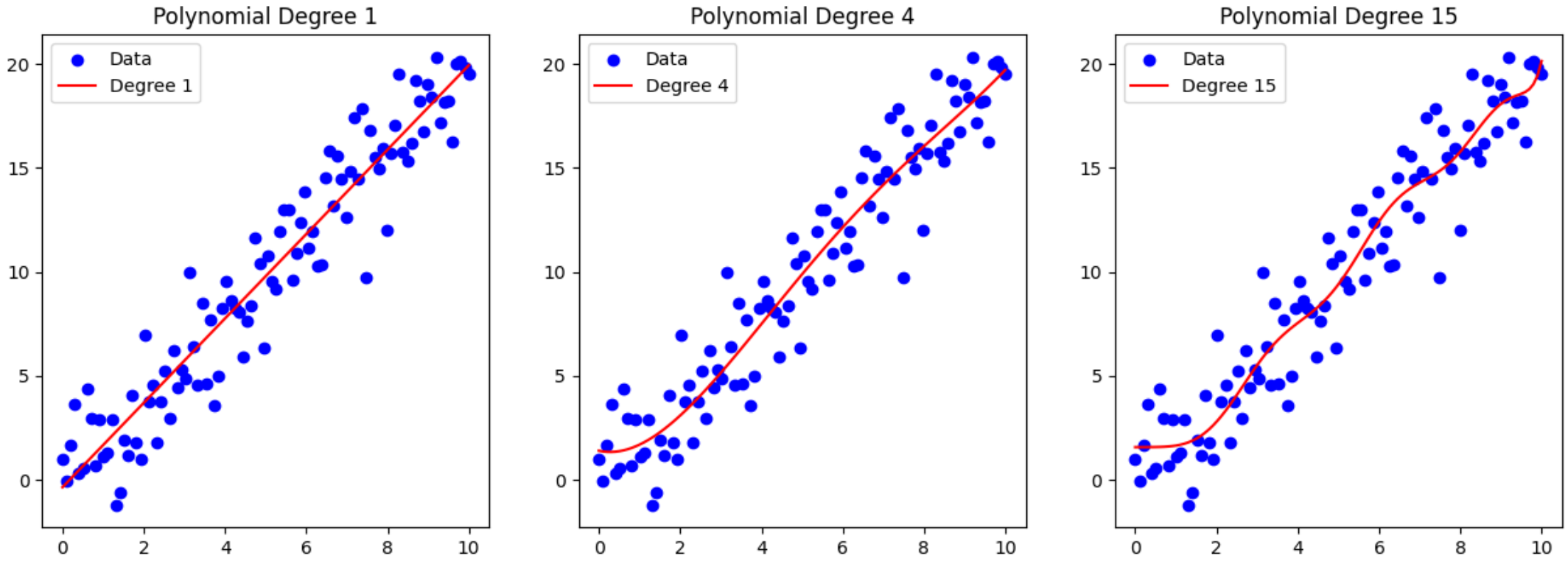
Tradeoff

The tradeoff arises because:

- Reducing bias typically increases variance (e.g., using a more complex model).
- Reducing variance typically increases bias (e.g., using a simpler model).

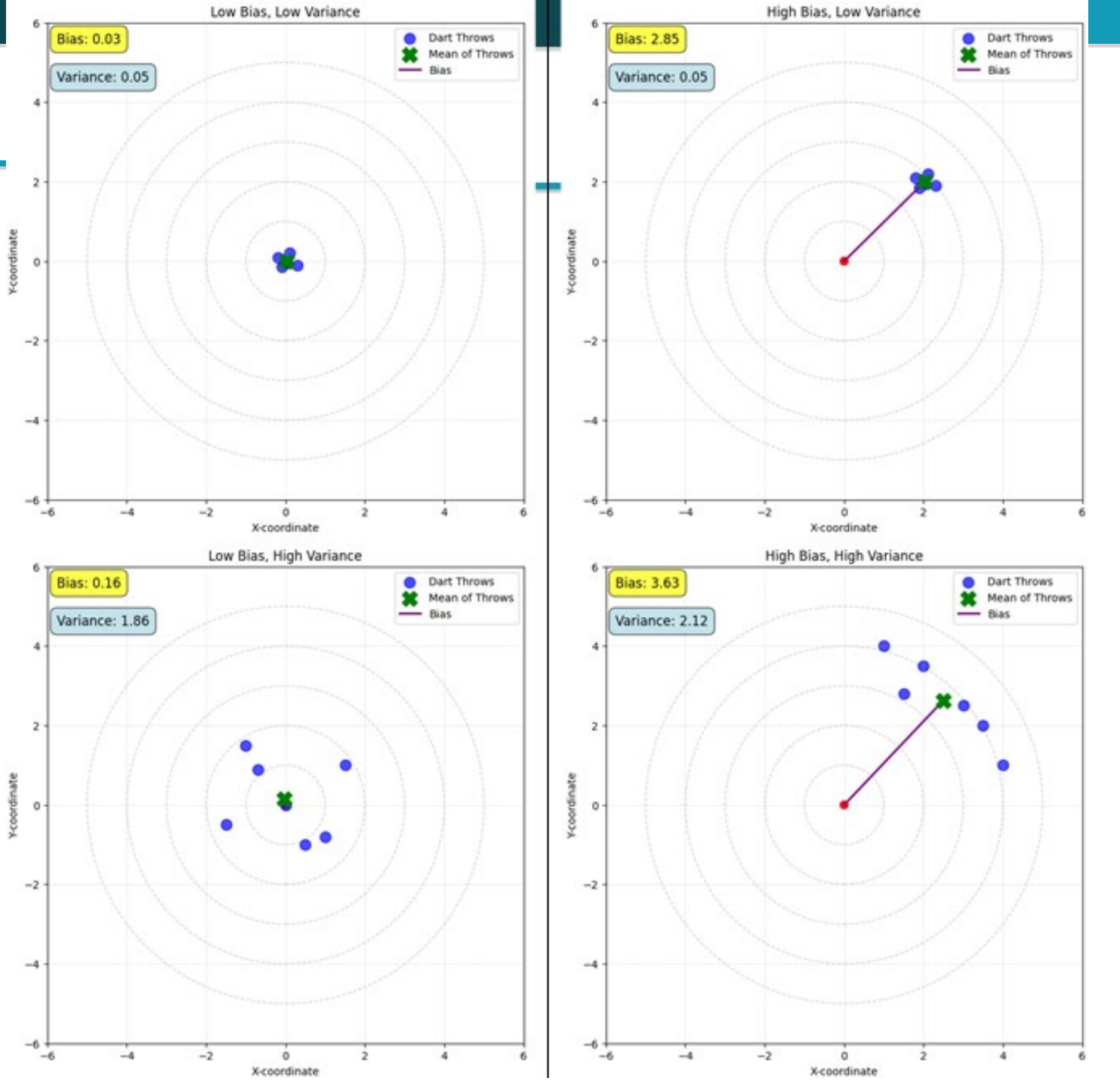
Finding the ideal ratio of bias to variance in order to reduce the overall error is the aim.

Visual Representation



- Degree 1: High bias (underfitting).
- Degree 4: Good fit (balanced bias and variance).
- Degree 15: High variance (overfitting).

Visual Representation

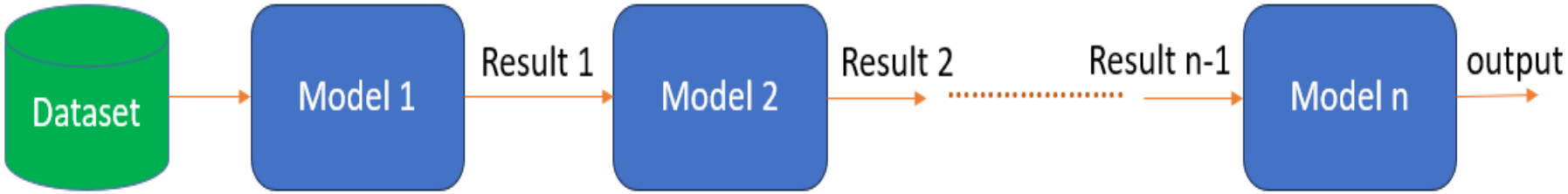


Types

- **Sequential Ensemble Methods**
  - Base learners are built sequentially, where each new learner tries to correct the mistakes of its predecessors
- **Parallel Ensemble Methods**
  - Base learners are built independently and concurrently. There is no dependency between the learners during the training phase.

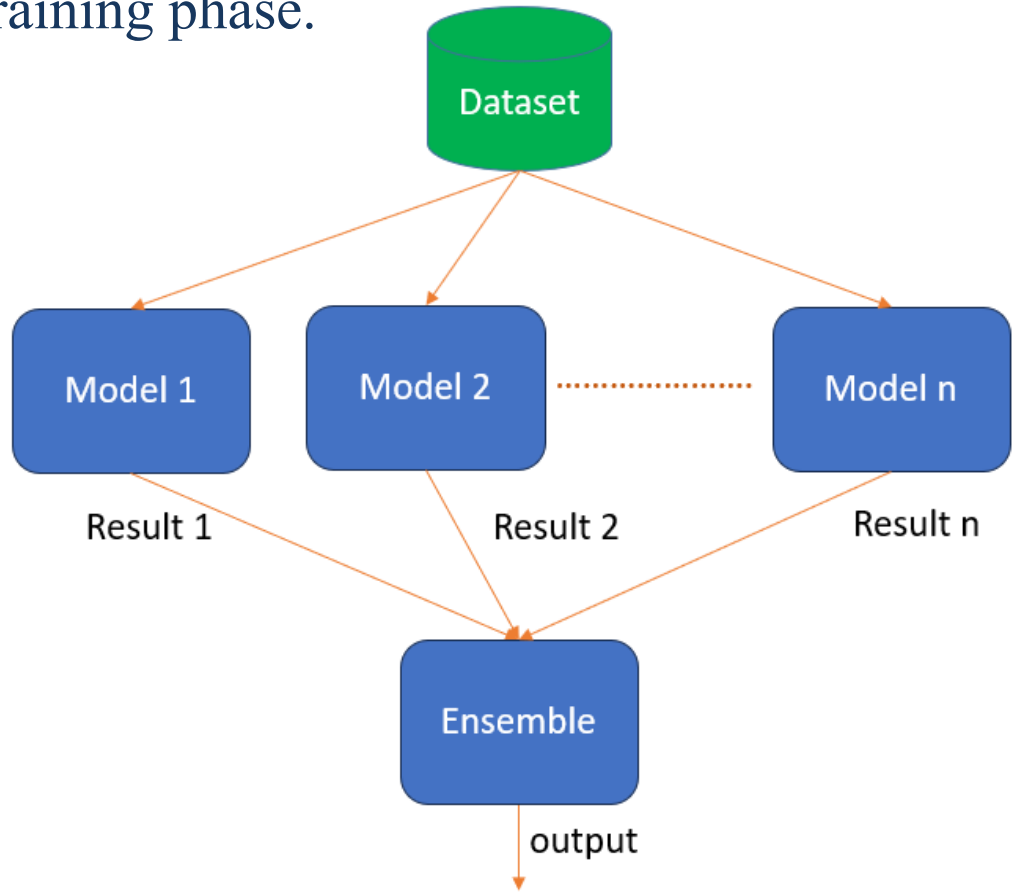
Types

- **Sequential Ensemble Methods**
  - Base learners are built sequentially, where each new learner tries to correct the mistakes of its predecessors



Types

- **Parallel Ensemble Methods**
  - Base learners are built independently and concurrently. There is no dependency between the learners during the training phase.



Types

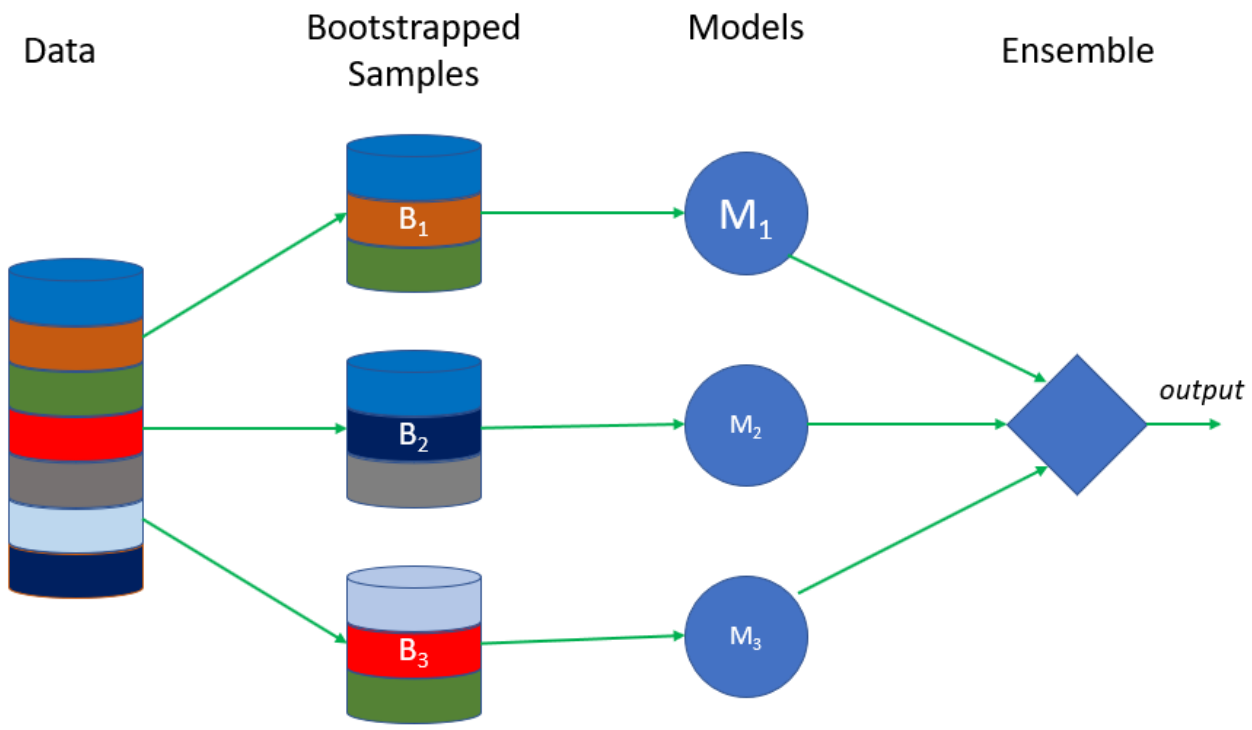
Feature	Sequential Ensemble Methods (e.g., AdaBoost, GBM)	Parallel Ensemble Methods (e.g., Bagging, Random Forest)
Training	Sequential, learners depend on each other	Parallel, learners are independent
Goal	Correct mistakes of previous learners	Create diverse learners and aggregate predictions
Dependency	High	None
Computation	Can be computationally expensive	Can be easily parallelized
Overfitting	More prone	Less prone
Typical Algorithms	AdaBoost, Gradient Boosting Machines (GBM)	Bagging, Random Forest

Common Ensemble Methods

1. Bagging (Bootstrap Aggregating)
2. Stacking (Stacked Generalization)
3. Boosting
4. Voting
5. Blending

Bagging (Bootstrap Aggregating)

Bagging trains multiple base learners by generating several fresh datasets from a single initial training dataset using a process known as bootstrap resampling.



Calculation Example:

Scenario: Predicting House Prices

Here's a sample dataset:

House ID	Square Footage (X_1)	Number of Bedrooms (X_2)	House Price (Y)
1	1000	2	200,000
2	1500	3	250,000
3	2000	3	300,000
4	2500	4	350,000
5	3000	4	400,000

Step 1

We generate **three bootstrapped datasets** by randomly selecting samples **with replacement**:

- **Dataset 1**: {(1,2,200K), (2,3,250K), (5,4,400K)}
- **Dataset 2**: {(2,3,250K), (3,3,300K), (4,4,350K)}
- **Dataset 3**: {(1,2,200K), (3,3,300K), (5,4,400K)}

Each dataset is used to train an **individual decision tree**.

Step 2: Train Models on Bootstrapped Data

Each Decision Tree learns relationships between **Square Footage** + **Bedrooms** and **House Price**.

Calculation Example:

House ID	Square Footage (X_1)	Number of Bedrooms (X_2)	House Price (Y)
1	1000	2	200,000
2	1500	3	250,000
3	2000	3	300,000
4	2500	4	350,000
5	3000	4	400,000

Step 3: Predictions from Individual Models

Let's assume our trained models predict the **house price** for **a new house (X_1 = 1800, X_2 = 3)**.

Model	Predicted Price
Model 1	260,000
Model 2	270,000
Model 3	255,000

Step 4: Aggregated Prediction (Bagging)

The final prediction is computed as the **average** of individual model predictions:

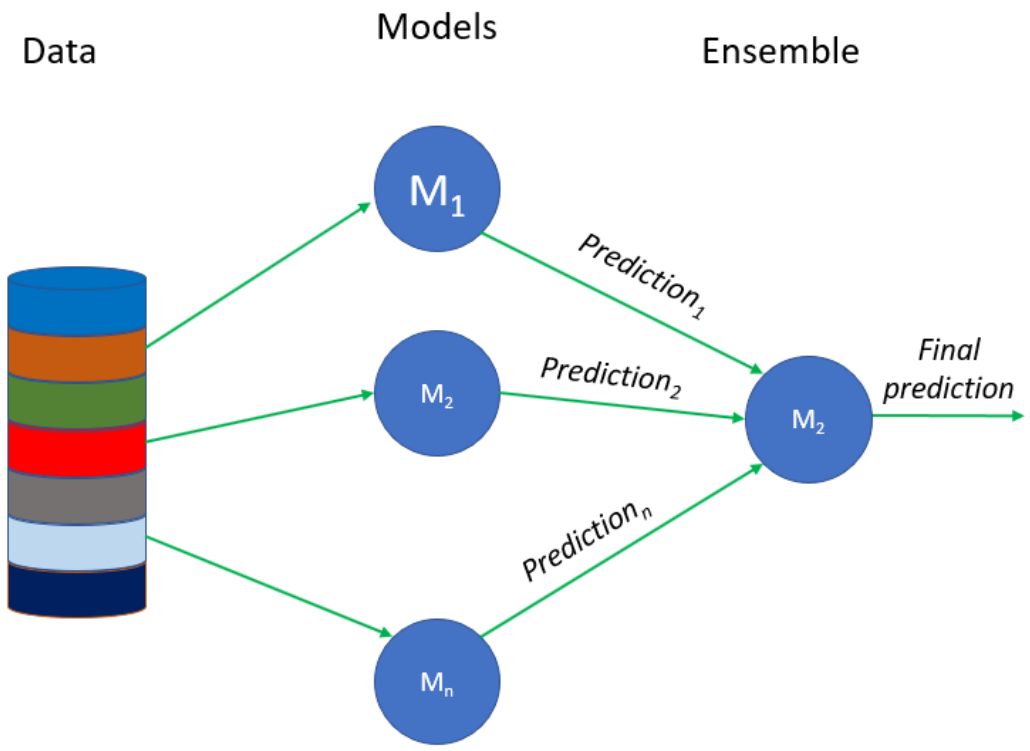
$$Y_{final} = \frac{(260{,}000 + 270{,}000 + 255{,}000)}{3}$$

$$Y_{final} = 261{,}667$$

So, the **aggregated prediction for the new house** is **$261,667**.

Stacking (Stacked Generalization)

Stacking uses a distinct training strategy for each basis learner to train several base learners from the same dataset. A final model, known as the meta-model, is trained using these initial model predictions.



Calculation Example:

Step 3: Train Meta-Learner

The meta-learner learns to combine predictions from base models to optimize accuracy.

House ID	Square Footage (X_1)	Number of Bedrooms (X_2)	House Price (Y)
1	1000	2	200,000
2	1500	3	250,000
3	2000	3	300,000
4	2500	4	350,000
5	3000	4	400,000

Using the base model predictions as input, we fit a meta-model:
Meta-Model Formula

Model	Predicted Price
Linear Regression (LR)	260,000
Decision Tree Regression (DT)	275,000
K-Nearest Neighbors (KNN) Regression (KNN)	255,000

$$Y_{final} = w_1 \cdot Y_{LR} + w_2 \cdot Y_{DT} + w_3 \cdot Y_{KNN} + b$$

Assuming optimal weights found via training:

w1 = 0.4 (Linear Regression contributes 40%)
w2 = 0.3 (Decision Tree contributes 30%)
w3 = 0.3 (KNN contributes 30%)
b = 5,000 (Bias term)

Step 4: Compute Final Prediction

$$Y_{final} = (0.4 \times 260{,}000) + (0.3 \times 275{,}000) + (0.3 \times 255{,}000) + 5{,}000$$

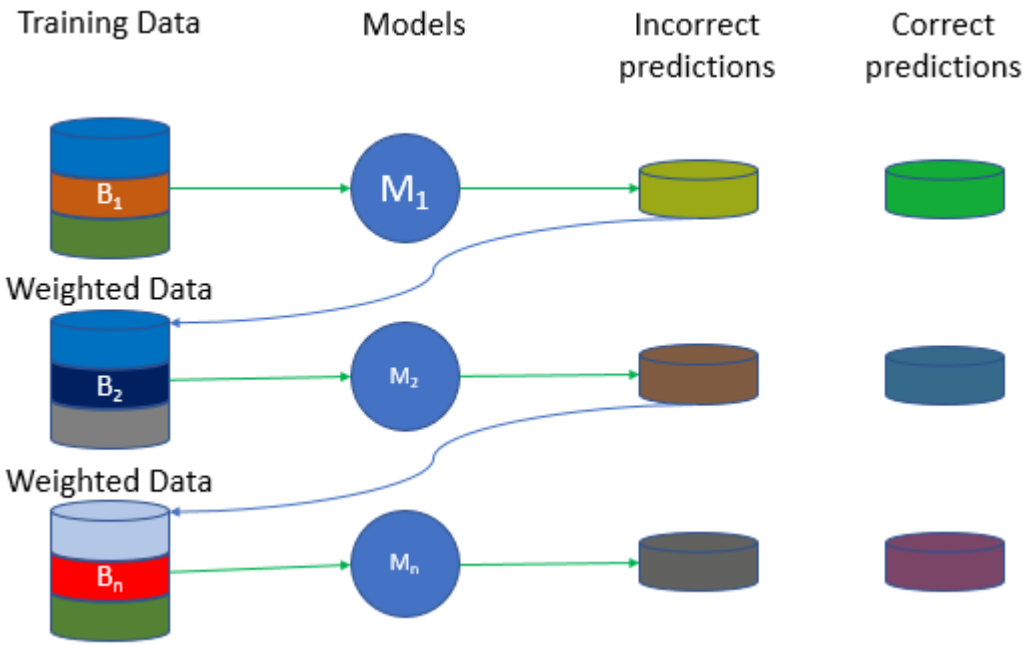
$$Y_{final} = 104{,}000 + 82{,}500 + 76{,}500 + 5{,}000$$

$$Y_{final} = 268{,}000$$

Boosting

Boosting improves model accuracy by iteratively training multiple weak learners.

- The process starts with an initial dataset (d) used to train a weak model, which misclassifies many instances. Unlike bagging, boosting prioritizes misclassified samples when generating a new dataset (d2), which is then used to train another learner.
- The third dataset (d3) gives further priority to samples where previous models disagreed.
- This iterative process continues for n learners, each refining the classification performance of the previous ones, ultimately resulting in a strong predictive model.



Calculation Example:

Scenario: Predicting House Prices

Here's a sample dataset:

House ID	Square Footage (X_1)	Number of Bedrooms (X_2)	House Price (Y)
1	1000	2	200,000
2	1500	3	250,000
3	2000	3	300,000
4	2500	4	350,000
5	3000	4	400,000

Step 1: Initialize Model (First Weak Learner)

The first weak model (e.g., a shallow decision tree) **learns on the full dataset** but makes mistakes. Let's assume predictions:

House ID	True Price(Y)	First Model Prediction ($\hat{y}_1$)	Error (y - $\hat{y}_1$)
1	200,000	220,000	**−20,000**
2	250,000	260,000	**−10,000**
3	300,000	290,000	**+10,000**
4	350,000	340,000	**+10,000**
5	400,000	380,000	**+20,000**

Step 2: Compute Sample Weights for Second Learner

Boosting **reweights samples** to focus more on **incorrect predictions**. Higher errors mean higher weights.
Let's assume a simple weighting scheme:

- **Higher error → Higher weight** in the next model.

House ID	Error (y - $\hat{y}_1$)	New Weight
1	**−20,000**	**1.5**
2	**−10,000**	**1.2**
3	**+10,000**	**1.2**
4	**+10,000**	**1.2**
5	**+20,000**	**1.5**

Calculation Example:

Step 3: Train Second Model on Weighted Data

A second model **focuses more** on **high-error samples** (e.g., House IDs 1 and 5). New predictions:

House ID	True Price(Y)	First Model Prediction ($\hat{y}_1$)
1	200,000	210,000 (improved)
2	250,000	255,000
3	300,000	295,000
4	350,000	345,000
5	400,000	390,000 (improved)

Step 4: Final Boosting Prediction (Weighted Sum of Weak Learners)
Boosting combines predictions using weights.

$$Y_{final} = w_1 \cdot Y_{Model1} + w_2 \cdot Y_{Model2}$$

Assume equal weight for models (w1 = 0.5, w2 = 0.5):

House ID	Model 1 Prediction ($\hat{y}_1$)	Model 2 Prediction ($\hat{y}_2$)	Final Boosting Prediction ($\hat{y}_{final}$)
1	220,000	210,000	215,000
2	260,000	255,000	257,500
3	290,000	295,000	292,500
4	340,000	345,000	342,500
5	380,000	390,000	385,000

Final Boosting Prediction

For a **new house (X_1 = 1800, X_2 = 3)**, predictions from models:

- Model 1 predicts **265,000**
- Model 2 predicts **275,000**
- **Boosting Prediction:**

$$Y_{final} = \frac{265{,}000 + 275{,}000}{2} = 270{,}000$$

Thus, the **boosted prediction for house price** with **X_1 = 1800, X_2 = 3** is **$270,000**.

Voting

- Voting refers to **collective decision-making** mechanisms whereby multiple models or components collaboratively determine the significance of features, thereby enhancing the robustness and reliability of feature selection.
- By aggregating the judgments of multiple models or learners, voting mitigates the biases and variances associated with individual models, leading to more stable and generalizable feature subsets

Common aggregation strategies include:

- **Averaging:** Computing the mean importance score for each feature across all models to obtain a smoothed estimate.
- **Weighted Voting:** Assigning differential weights to models based on their predictive performance, reliability, or confidence, thereby emphasizing contributions from more accurate models.
- **Majority Voting:** Selecting features that are deemed important by the majority of models, effectively applying a threshold based on collective agreement.

Calculation Example:

Scenario: Predicting House Prices

Here's a sample dataset:

House ID	Square Footage (X_1)	Number of Bedrooms (X_2)	House Price (Y)
1	1000	2	200,000
2	1500	3	250,000
3	2000	3	300,000
4	2500	4	350,000
5	3000	4	400,000

Step 1: Predictions from Individual Models

For a **new house with square footage = 1800** and **number of bedrooms = 3**, each model provides a price estimate:

Model	Predicted Price
Linear Regression (LR)	265,000
Decision Tree Regression (DT)	275,000
K-Nearest Neighbors (KNN)	260,000

Step 2: Apply Voting Ensemble

Since we are dealing with **regression**, we use **averaging** to combine predictions:

$$Y_{final} = \frac{(265{,}000+275{,}000+260{,}000)}{3}$$

$$Y_{final} = \frac{800{,}000)}{3} = 266{,}667$$

Thus, the **final predicted house price** using the **Voting Ensemble Method** is **$266,667**.

Blending

Blending enhances predictive performance by combining outputs from multiple individual models.

Blending serves a dual purpose: integrates the feature selection process directly into the model training framework while simultaneously optimizing model predictions through the systematic aggregation of multiple base learners.

Steps:

1. Feature Selection via Base Models,
2. Prediction Generation,
3. Stacking or Aggregation of Predictions,
4. Feature Relevance Aggregation.

Calculation Example:

Scenario: Predicting House Prices

Here's a sample dataset:

House ID	Square Footage (X_1)	Number of Bedrooms (X_2)	House Price (Y)
1	1000	2	200,000
2	1500	3	250,000
3	2000	3	300,000
4	2500	4	350,000
5	3000	4	400,000

Step 1: Predictions from Individual Models

For a **new house with square footage = 1800** and **number of bedrooms = 3**, each model provides a price estimate:

Model	Predicted Price
Linear Regression (LR)	265,000
Decision Tree Regression (DT)	275,000
K-Nearest Neighbors (KNN)	260,000

Step 2: Create Validation Set & Train Meta-Model

- A **validation dataset** is generated where each sample consists of predictions from the base models.
- The meta-model learns how to **combine** the predictions optimally.

Meta-Model Formula

$$Y_{final} = w_1 \cdot Y_{LR} + w_2 \cdot Y_{DT} + w_3 \cdot Y_{KNN} + b$$

where:

- w_1 = 0.5 (Linear Regression contributes 50%)
- w_2 = 0.3 (Decision Tree contributes 30%)
- w_3 = 0.2 (KNN contributes 20%)
- b = 5,000 (Bias term)

Calculation Example:

Model	Predicted Price
Linear Regression (LR)	265,000
Decision Tree Regression (DT)	275,000
K-Nearest Neighbors (KNN)	260,000

Step 3: Compute Final Blended Prediction

$$Y_{final} = (0.5 \times 265{,}000) + (0.3 \times 275{,}000) + (0.2 \times 260{,}000) + 5{,}000$$

$$Y_{final} = 132{,}500 + 82{,}500 + 52{,}000 + 5{,}000$$

$$Y_{final} = 272{,}000$$

Thus, the **final predicted house price** using **Blending Ensemble Method** is **$272,000**.